

SHIELD: PSO-Based Hardware Trojan Detection for Efficient and Low-Cost Defense

Mostafa Hosseini¹, Ali Azarpeyvand^{1,2}, Mahdi Taheri², Tara Ghasempouri², and Maksim Jenihhin²

¹University of Zanjan, Zanjan, Iran

²Tallinn University of Technology, Tallinn, Estonia

¹{mostafa.hosseini, azarpeyvand}@znu.ac.ir

²{ali.azarpeyvand, mahdi.taheri, tara.ghasempouri, maksim.jenihhin}@taltech.ee

Abstract—Semiconductor supply chain vulnerability presents a significant obstacle to creating reliable systems. At various phases of the Integrated Circuit (IC) design life-cycle, malicious modifications, known as Hardware Trojans (HTs), can be introduced. Logic testing, a widely recognized approach for Automatic test pattern Generation (ATPG) in HT detection, encounters substantial challenges due to the vast complexity of the search space, making it impractical and leading to inadequate trigger coverage. This paper proposes a Particle Swarm Optimization (PSO) based method that leverages information on effective inputs to facilitate the detection of conditionally triggered ultra-small HTs. An evaluation of the technique on ISCAS-85 benchmarks reveals substantial improvements in trigger coverage and a notable reduction in runtime compared to state-of-the-art methods.

Index Terms—Hardware Trojan, Test Generation, Particle Swarm Optimization, Depth-First Search.

I. INTRODUCTION

The growing dependence on third-party entities for the design and fabrication of ICs has substantially increased their exposure to security threats. Among these, HTs—malicious modifications embedded during design or manufacturing—are particularly insidious, comprising a trigger mechanism and a payload that activates under specific conditions. The stealthy nature of such Trojans, especially those with rare or conditional triggers, renders traditional simulation-based validation approaches ineffective, as they rely on extensive test vector application and struggle with the exponential complexity of modern IC input spaces [1]. This has led to an urgent demand for high trigger coverage and scalable detection methodologies [2].

Initial efforts in logic testing focused on random or constrained-random ATPG due to their implementation simplicity; however, these approaches offer no guarantee of activating stealthy HTs [3]. To improve coverage, MERO [4] introduced a statistical ATPG technique based on the N-detect concept, aiming to activate rare signals multiple times. Although effective for small benchmarks, the brute-force nature of this method limits its scalability. Genetic Algorithm (GA)-based approaches, such as TRIAGE [5], leverage transition probabilities and rare node analysis to generate targeted test patterns. Despite their improved detection potential, these

methods often encounter prolonged execution times and convergence issues during later evolutionary stages. PSO-based methods [6] offer the advantage of rapid convergence in early search phases but are highly sensitive to algorithm parameters and initial population configurations. Subsequent GA-based strategies [7] have addressed these challenges by integrating structural characteristics and SCOAP metrics into fitness evaluations, enabling the detection of rare, conditionally triggered Trojans with reduced computational overhead. The most recent advancement in this domain is the PATROL algorithm [2], which synergistically combines PSO's local search efficiency with GA's global optimization capability. This hybrid method achieves a 43-fold reduction in test generation time and a 34% improvement in trigger coverage, marking a significant progression in the field of HT detection.

This work investigates a threat model where adversaries insert combinational HTs at the gate-level during design or fabrication. The HT trigger employs a 2-input AND gate that activates a payload upon detecting a rare logic '1', triggered by specific input patterns. This paper presents SHIELD, a cost-effective HT inspection using PSO-based test pattern generation. The principal contributions of this work are as follows:

- **Efficient and Low-Cost PSO-based ATPG:** A PSO-based approach has been introduced, meticulously tailored to address selective inputs. These inputs have been identified based on their significant influence on rare nets.
- **Search Space Reduction:** The Depth First Search (DFS) algorithm is utilized in graph traversal to assess the impact of primary inputs (PIs) on rare nets, thereby reducing the search space by identifying and selecting the most effective inputs.
- **ATPG Framework For HT Detection:** An open-source framework is developed for HT detection at gate-level circuits <https://github.com/mostafahosseini72/ShEILD>.

The remainder of the paper is organized as follows: Section II provides an overview of the proposed method. Section III reports the experimental results. Finally, Section IV concludes the paper.

II. METHODOLOGY

The proposed methodology encompasses three well-defined stages, as illustrated in fig 1. Initially, rare nodes are identified,

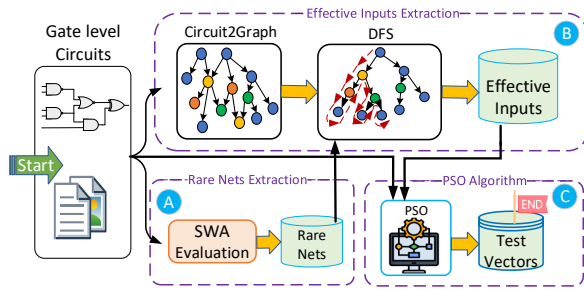


Fig. 1: SHIELD Three-Step HT Detection Workflow.

constituting the first step. Subsequently, effective inputs corresponding to the identified rare nodes are extracted, thereby substantially reducing the search space, which forms the second step. Finally, the PSO algorithm is applied to the target circuit, leveraging the effective inputs obtained in Step 2 to generate optimized test vectors, completing the third step.

A. Rare Nets Extraction

A low-activity net (Net_i) has a switching rate (α) significantly lower than other nets in the circuit. Switching Activity (SWA) is defined as the ratio of a net's transitions to the total number of input states, determined by evaluating its behavior across all 2^n input patterns. A transition occurs when a net's logic state changes between two consecutive vectors. Identical test vectors (e.g., v_1 and v_2) cause no transitions, while differing ones can. Nets with very low SWA typically stay at a constant logic level ('0' or '1'). To detect such nets, the circuit is simulated using a moderate number of random test vector pairs (e.g., 1000), and those with infrequent transitions beyond a set threshold are flagged for further analysis.

B. Effective Inputs Extraction

The gate-level logic circuit is converted into a directed graph ($DG = (V, E)$) using the NetworkX Python library, where vertices V represent logic gates and edges E represent nets. The circuit is then leveled, and rare nets identified earlier are analyzed using Depth-First Search (DFS). This analysis identifies input combinations that activate the rare nets, while other inputs are deemed non-contributory, thereby reducing the search space. Simulation results show that for circuits c880, c2670, and c5315—with 60, 233, and 178 PIs, respectively—only 23, 148, and 62 inputs are effective for rare nets with $SWA < 0.1$. This leads to a search space reduction of 2^{37} , 2^{85} , and 2^{116} , respectively. The DFS-based input tracing algorithm is designed to identify the PI nodes that influence specific rare nets within a digital circuit represented as a directed graph. For each rare net in the set, the algorithm performs a DFS traversal in reverse—moving backward through the graph from the rare net toward the PIs. During this traversal, whenever an input node is encountered, it is recorded along with its index from the predefined list of input nodes. The algorithm compiles a dictionary that maps each rare net to the set of input nodes that can affect it, providing valuable insight into controllability and potential signal propagation paths relevant for tasks such as test pattern generation or HT detection.

C. Particle Swarm Optimization

In this approach, each pair of test vectors (v_1, v_2) is constructed by initializing v_1 as an array of zeros, while v_2 is derived through the PSO algorithm. The PSO process begins by initializing particle positions and velocities using randomly generated test vectors, which are subsequently guided by a fitness function. During each iteration, particles are evaluated and ranked according to their fitness values, determined by the number of rare nets they activate within combinational circuits. The optimization proceeds iteratively, with high-performing vectors propagated as initial candidates for subsequent iterations. A subset of top-performing individuals is preserved to track globally optimal solutions, with the overarching goal of maximizing rare net coverage while minimizing the number of required test vectors. The PSO-based test vector generation strategy employed in the SHIELD system is detailed in Algorithm 1.

The algorithm begins by setting essential parameters and initializing the swarm (Lines 1–10). Subsequently, a set of random patterns is generated to facilitate initial fitness evaluation (Line 11). Rare gates are identified within the circuit (Line 12), and tracking structures are initialized to monitor optimization progress (Line 13). Input signals are then ranked based on their impact, ensuring prioritization of the most effective ones (Line 14), followed by the initialization of particle-specific attributes (Line 15). The global best position—which serves as a reference point to guide the swarm's movement—is computed (Line 16). The iterative process continues until the predefined threshold for rare gate coverage is met (Line 17). A critical parameter within the PSO framework is the inertia weight (IW), which regulates the balance between global exploration and local exploitation. The IW value decreases linearly from a predefined maximum to a minimum throughout

Algorithm 1 PSO-Based Test Vector Generation

```

1: Input: effective_inputs
2: Output: test_vectors
3: function PSO(effective_inputs)
4:    $max\_iterations \leftarrow 100$ 
5:    $num\_particles \leftarrow 150$ 
6:    $IW\_max \leftarrow 0.9$ 
7:    $IW\_min \leftarrow 0.4$ 
8:    $cognitive\_weight \leftarrow 2.4$ 
9:    $social\_weight \leftarrow 2.0$ 
10:  Zero Initialize  $swarm\_position, fitness\_values$ 
11:  Generate random test patterns and evaluate fitness
12:  Identify  $rare\_nets$  and compute  $total\_rare\_nets$ 
13:  Initialize an empty list  $activated\_rare\_nets,$ 
     $test\_vectors, TTVG$ 
14:  Sort effective_inputs by impact on rare nets
15:  Zero Initialize  $particle\_positions, particle\_velocities,$ 
     $particle\_best\_positions, particle\_best\_fitness$ 
16:  Compute  $Gbest\_position$  and  $Gbest\_fitness$ 
17:  while  $|activated\_rare\_nets| < \text{desired Coverage}$  do
18:     $TTVG \leftarrow TTVG + 1$ 
19:    Dynamic parameter adjustment
20:    for  $iteration = 1, 2, \dots, max\_iterations$  do
21:      Update velocities and positions
22:      Evaluate fitness values for new positions
23:      Update personal best and global best positions
24:    end for
25:  end while
26:  return  $test\_vectors$  and  $TTVG$ 
27: end function

```

the execution, thereby promoting broad exploration during the initial phases and facilitating precise convergence in the later stages (Line 19), as formalized in Equation 1.

$$IW = IW_{\max} - \frac{(IW_{\max} - IW_{\min})}{\max_iterations} \times iterations \quad (1)$$

where IW represents the current inertia weight at each step of the algorithm, $IW_{\max}$ denotes the maximum inertia weight at the beginning of the algorithm, $IW_{\min}$ refers to the minimum inertia weight at the end of the algorithm, iteration is the count of iterations or steps (ranging from 0 to $\max_iterations$), and $\max_iterations$ indicates the total number of iterations (or total number of steps) in the optimization process.

Particles can update their positions (Line 21) by equation 2:

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (2)$$

Where x_i represents a randomly initialized position, and v_i represents a randomly initialized velocity. The velocity of the random particle is determined by the formula outlined in equation 3.

$$v_i(t+1) = \omega v_i(t) + c_1 r_1 (p_i^{\text{best}} - x_i(t)) + c_2 r_2 (g^{\text{best}} - x_i(t)) \quad (3)$$

Here, p_i^{best} represents the best particle position, g^{best} denotes the best global position, ω is the inertia weight, c_1 and c_2 are constants, and r_1 and r_2 are random parameters within the range [0, 1]. Since r_1 and r_2 must fall within the range [0, 1], the sigmoid function is used. In PSO, the sigmoid transformation maps velocity data to a restricted range between 0 and 1, enabling non-linear adjustments that balance exploration and exploitation. For binary conversion of fractional values, the rand function from the numpy library is used, applying a 0.5 threshold to assign values of 0 or 1. Elements below 0.5 are set to 0, while those at or above 0.5 are set to 1.

This function is applied to adjust the velocity data, as indicated by the following statement. To prevent sudden changes in particle positions, we restrict their velocity according to Equation 4, typically within 10-20% of the variable range. This velocity clamping mechanism helps to mitigate extreme position shifts.

$$v_i(t+1) = \text{clip}(v_i(t+1), -v_{\max}, v_{\max}) \quad (4)$$

where $v_i(t+1)$ is the Updated velocity of particle i at iteration $t+1$ and $v_{\max}$ is Maximum allowed velocity.

After velocities are updated, particles move to new positions and evaluate their fitness value (Line 22). The fitness value starts at zero, and rare nets are checked for their unique outputs. If these outputs match the rarity criteria, the fitness value increases. Update the particle's position if this new fitness surpasses the personal best (Line 23). By comparing the personal best positions of all the particles in the swarm and selecting the one with the highest fitness value, the global best position can be identified. The list called "best individuals" is now sorted by fitness values, allowing us to identify the top individuals (Lines 26).

III. EXPERIMENTAL RESULTS

All experiments use a system with an Intel(R) Core(TM) i7-7700HQ CPU @ 3.80GHz, 3792 MHz, 4 Core(s), 8 Logical Processor(s), 16GB of RAM, and a 64-bit Windows 10 operating system. In the experimental analysis, rare nets (HT trigger nets) are identified by employing two distinct activation thresholds (α_{th}) of 0.1 and 0.2. We conducted a comparative analysis between our proposed method and the five approaches introduced in [4], [5], [6], [7] and [2]. Henceforth, these methods will be referred to as MERO, TRIAGE, M1, M2 and PATROL respectively. The primary distinction between SHIELD and the M1 method lies in the integration of advanced optimizations, such as Dynamic Parameter Tuning and Velocity Clamping, coupled with a significant reduction of the search space through the application of the DFS algorithm. Furthermore, the effective inputs are prioritized by sorting them based on the frequency of their impact on the existing rare nets, ensuring that the test vector generation process is conducted in accordance with this prioritization.

Table I provides a comparison of trigger coverage and runtime, measured in seconds, against the existing approaches for each circuit in the ISCAS-85 benchmark circuits [8] (CKTs INSTANCE).

Runtime Improvement: The SHIELD method significantly improves computational speed, outperforming MERO, TRIAGE, M1, M2, and PATROL by over 73×, 15×, 17×, 12×, and 9× on average, respectively. Notably, SHIELD achieves up to a 197× speedup over MERO for the C6288 circuit, with a minimum of 7× for C2670. However, high trigger coverage was not attainable for C6288 due to its equal number of primary and effective inputs. Against TRIAGE, SHIELD reaches a maximum 37× speedup (C6288) and a minimum 2× (C1908). For M1, it achieves up to 37× (C6288) and at least 2× (C1908); for M2, up to 30× (C880) and as low as 1.2× (C2670). These results show that test vector generation time depends on circuit size, topology, node SWA thresholds, and the initial parameters of the optimization algorithms. Performance can be further improved by narrowing the search space, tuning PSO parameters, and increasing node SWA thresholds.

Improvement in trigger coverage: The proposed method SHIELD demonstrates a significant improvement in Trigger Coverage, surpassing the MERO, TRIAGE, M1, M2, and PATROL methods by more than 44%, 23%, 19%, 13%, and 6%, respectively. Unsurprisingly, MERO performs poorly due to its brute-force bit-flipping strategy. Specifically, compared to the MERO method, the proposed approach achieves an approximate 64% improvement in optimal coverage for the C3540 circuit, with a minimum coverage enhancement of 32% for the C7552 circuit. While TRIAGE decline stems from limitations of employing a genetic algorithm in this context, the search process is unguided and lacks direction, and the algorithm is highly sensitive to initial population and parameter configurations. Furthermore, the GA-based proposed approach requires significantly more time to generate

TABLE I: Comparison of Trigger Coverage and Runtime with existing approaches

CKTs INSTANCE	PIs	α_{Th}	Effective Inputs	Total Rare Gate	MERO		TRIAGE		M1		M2		SHIELD	
					Trigger Cov.(%)	Time (Sec)	Trigger Cov.(%)	Time (Sec)	Trigger Cov.(%)	Time (Sec)	Trigger Cov.(%)	Time (Sec)	Trigger Cov.(%)	Time (Sec)
C880	60	0.1	23	48	45	154.31	76	71.00	79	51.36	89	57.73	100	1.85
		0.2	36	28	64	155.10	84	68.74	87	42.83	97	49.92	100	1.99
C1908	33	0.1	33	104	50	149.00	82	39.28	65	42.02	94	32.34	100	12.48
		0.2	10	9	65	152.00	89	40.10	78	76.43	98	34.37	100	2.80
C1355	41	0.1	41	105	50	178.00	79	31.10	86	67.76	88	27.73	100	9.48
		0.2	41	9	57	232.97	94	29.63	100	52.91	94	23.65	100	2.30
C2670	233	0.1	148	78	50	553.72	58	207.43	65	289.73	76	169.36	88	75.00
		0.2	94	121	67	566.82	69	189.73	76	247.42	89	142.83	100	16.21
C3540	50	0.1	48	257	35	789.66	53	169.10	68	229.93	58	131.47	99	10.55
		0.2	49	222	50	785.23	79	159.53	76	172.76	74	127.73	100	6.35
C5315	178	0.1	62	46	45	1542.68	65	331.95	73	369.65	79	298.34	97	20.53
		0.2	98	155	54	1468.89	69	329.52	79	352.67	86	285.28	100	23.99
C6288	32	0.1	32	68	47	1661.39	53	269.21	62	324.67	58	249.73	74	8.36
		0.2	32	195	56	1724.48	67	257.57	78	316.87	74	240.86	100	10.99
C7552	207	0.1	198	155	52	3200.33	67	542.87	79	723.86	79	496.05	87	101.00
		0.2	197	143	64	3200.33	89	532.97	87	683.87	96	482.87	95	37.96
Average	104.25		71.38	108.94	53.19	1032.18	73.31	204.36	76.80	265.13	83.06	178.14	96.25	21.37

tests compared to alternative methods and is computationally intensive. When compared to the TRIAGE method, SHIELD delivers a best-case coverage improvement of approximately 46% for the C3546 circuit and, in the worst-case scenario, coverage enhancement of 6% for the C1355 circuit.

Table II presents the comparison of Trigger Coverage and Runtime with PATROL. Compared to the PATROL and PSO-GA Hybrid Algorithm method, SHIELD achieves approximately 19% higher trigger coverage in the optimal case for the C7552 circuit while maintaining the same minimum trigger coverage as PATROL. However, SHIELD significantly outperforms PATROL 64 times faster in the optimal case for the C880 circuit and exhibits a minimum speedup of equal for the C2670 circuit. The simulation results demonstrate that the impact of dimensionality reduction and parameter fine-tuning exceeds that of the initial population with a high fitness function, which is obtained through the Patrol method by integrating PSO and GA.

IV. CONCLUSIONS

This research introduces SHIELD, an enhanced PSO algorithm specifically tailored to operate on a subset of inputs selected based on their influence on rare nets, for the automated generation of test vectors aimed at detecting HTs. The primary aim of SHIELD is to address the challenges posed by the vast search space. The effectiveness of the proposed approach has been validated through a comparative analysis

TABLE II: Trigger Coverage and Runtime vs. PATROL

CKTs INSTANCE	α_{Th}	PATROL		SHIELD		Improv./PATROL	
		Trigger Cov.(%)	Time (Sec)	Trigger Cov.(%)	Time (Sec)	Trigger Cov.	Runtime
C880	0.1	100	18.31	100	1.85	0.00	8.90x
	0.2	100	129.93	100	1.99	0.00	64.29x
C1908	0.1	85	42.21	100	12.48	15.00	2.38x
	0.2	100	5.21	100	2.80	0.00	0.86x
C1355	0.1	100	84.43	100	9.48	0.00	7.91x
	0.2	100	9.87	100	2.30	0.00	3.29x
C2670	0.1	75	71.76	88	75.00	14.77	0.04x
	0.2	100	40.84	100	16.21	0.00	1.52x
C3540	0.1	86	64.42	99	10.55	13.13	5.11x
	0.2	95	174.98	100	6.35	5.00	26.56x
C5315	0.1	80	117.97	97	20.53	17.53	4.75x
	0.2	95	139.9	100	23.99	5.00	4.83x
C6288	0.1	70	66.82	74	8.36	5.41	6.99x
	0.2	95	59.98	100	10.99	5.00	4.46x
C7552	0.1	70	627.56	87	101.00	19.54	5.21x
	0.2	90	77.56	95	37.96	5.26	1.04x
Average		90.06	108.23	96.25	21.37	6.60	9.26x

with the MERO, TRIAGE, M1, M2 and PATROL algorithms. the experimental findings indicate a substantial reduction in test generation time, achieving improvements of approximately 73, 15, 17, 12, and 9 times, respectively. Additionally, there was a significant improvement in trigger coverage, surpassing the MERO, TRIAGE, M1, M2, and PATROL methods by over 44%, 23%, 19%, 13%, and 6%, respectively. Simulation results indicate that genetic and PSO metaheuristic algorithms are highly sensitive to initial parameter settings. Furthermore, narrowing the search space significantly enhances speed, particularly when achieving coverage below 100 percent with an SWA threshold of less than 0.1. Additionally, a notable acceleration is observed at higher coverage levels with a SWA threshold exceeding 0.1.

ACKNOWLEDGMENT

This work was supported in part by the Estonian Research Council grants PSG837.

REFERENCES

- [1] Hossein Rostami, Mostafa Hosseini, Ali Azarpeyvand, Mohammad Reza Heidari Iman, and Tara Ghasempouri. Automatic high functional coverage stimuli generation for assertion-based verification. In *2024 IEEE 30th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, pages 1–7. IEEE, 2024.
- [2] Mostafa Hosseini, Ali Azarpeyvand, and Tara Ghasempouri. Patrol: An evolutionary approach to automatic test pattern generation for hardware trojan detection leveraging pso-ga hybrid techniques. In *2024 IEEE 33rd Asian Test Symposium (ATS)*, pages 1–6, 2024.
- [3] Aruna Jayasena and Prabhat Mishra. Directed test generation for hardware validation: A survey. *ACM Computing Surveys*, 56(5):1–36, 2024.
- [4] Rajat Subhra Chakraborty, Francis Wolff, Somnath Paul, Christos Papachristou, and Swarup Bhunia. Mero: A statistical approach for hardware trojan detection. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 396–410. Springer, 2009.
- [5] MA Nourian, Mahdi Fazeli, and David Hély. Hardware trojan detection using an advised genetic algorithm based logic testing. *Journal of Electronic Testing*, 34:461–470, 2018.
- [6] Sandip Chakraborty, Mudumba Sri Varenaya, Anindan Mondal, and Bibhash Sen. Exploring the pso-driven test pattern generation approach for hardware trojan detection. In *2024 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, volume 2, pages 1–6. IEEE, 2024.
- [7] Sandip Chakraborty, Archisman Ghosh, Anindan Mondal, and Bibhash Sen. Towards the detection of hardware trojans with cost effective test vectors using genetic algorithm. *Journal of Electronic Testing*, 40(3):371–385, 2024.
- [8] David Bryan. The iscas’85 benchmark circuits and netlist format. *North Carolina State University*, 25:39, 1985.